实验八 微服务的容错机制 Sentinel

一、 目的与要求

理解服务熔断技术、掌握 Sentinel 组件的使用以及使用 Sentinel 实现服务的限流与熔断

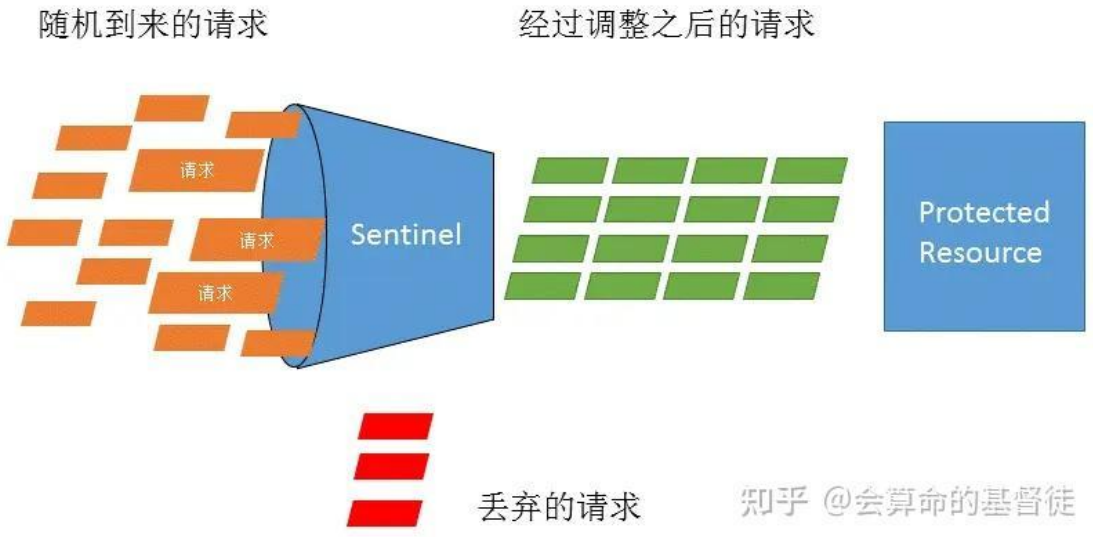
二、 实验内容

2.1 下载、安装、运行 Sentinel
2.2 使用 Sentinel 保护微服务

三、 实验指导

3.1 流量控制

流量控制在网络传输中是一个常用的概念，它用于调整网络包的发送数据。然而，从系统稳定性角度考虑，在处理请求的速度上，也有非常多的讲究。任意时间到来的请求往往是随机不可控的，而系统的处理能力是有限的。我们需要根据系统的处理能力对流量进行控制。Sentinel 作为一个调配器，可以根据需要把随机的请求调整成合适的形状，如下图所示：



3.2 Sentinel 介绍

（1）Sentinel 官网
（2）Sentinel 简介

Sentinel 是面向分布式、多语言异构化服务架构的流量治理组件，主要以流量为切入点，从流量控制、流量路由、熔断降级、系统自适应保护等多个维度来帮助用户保障微服务

的稳定性。

（3）Sentinel 特征

- 丰富的应用场景

Sentinel 承接了阿里巴巴近 10 年的双十一大促流量的核心场景，例如秒杀（即突发流量控制在系统容量可以承受的范围）、消息削峰填谷、集群流量控制、实时熔断下游不可用应用等。

- 完备的实时监控

Sentinel 同时提供实时的监控功能。您可以在控制台中看到接入应用的单台机器秒级数据，甚至 500 台以下规模的集群的汇总运行情况。

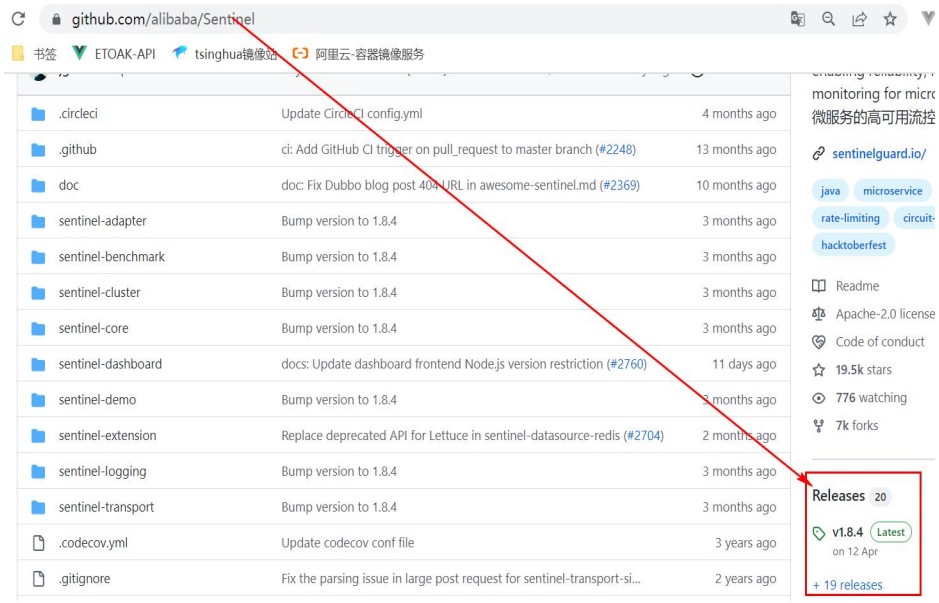
- 广泛的开源生态

Sentinel 提供开箱即用的与其它开源框架/库的整合模块，例如与 Spring Cloud、Apache Dubbo、gRPC、Quarkus 的整合。您只需要引入相应的依赖并进行简单的配置即可快速地接入 Sentinel。同时 Sentinel 提供 Java/Go/C++ 等多语言的原生实现。

- 完善的 SPI 扩展机制

Sentinel 提供简单易用、完善的 SPI 扩展接口。您可以通过实现扩展接口来快速地定制逻辑。例如定制规则管理、适配动态数据源等。

3.3 下载 Sentinel



3.4 运行 Sentinel

（1）.在 Sentinel 目录中执行：java -jar sentinel

```
D:\etoak_ws\soft\7.Cloud_Alibaba>java -jar sentinel-dashboard-1.8.2.jar
INFO: Sentinel log output type is: file
INFO: Sentinel log charset is: utf-8
INFO: Sentinel log base directory is: C:\Users\etoak\logs\csp\
INFO: Sentinel log name use pid is: false

 .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::        (v2.0.5.RELEASE)
```

（2）浏览器访问：localhost:8080

（3）输入用户名和密码：sentinel/sentinel

（4）在 Worker、Archive、Engineer 模块中，在 pom.xml 中引入 sentinel 依赖

```
<dependencies>
   <dependency>
     <groupId>org.springframework.boot</groupId>
```

```
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
      <groupId>com.alibaba.cloud</groupId>
      <artifactId>spring-cloud-starter-alibaba-nacos-discovery</artifactId>
    </dependency>

    <dependency>
      <groupId>com.alibaba.cloud</groupId>
      <artifactId>spring-cloud-starter-alibaba-sentinel</artifactId>
    </dependency>
  </dependencies>
```

（5）在 bootstrap.yml 中配置 sentinel 控制台地址

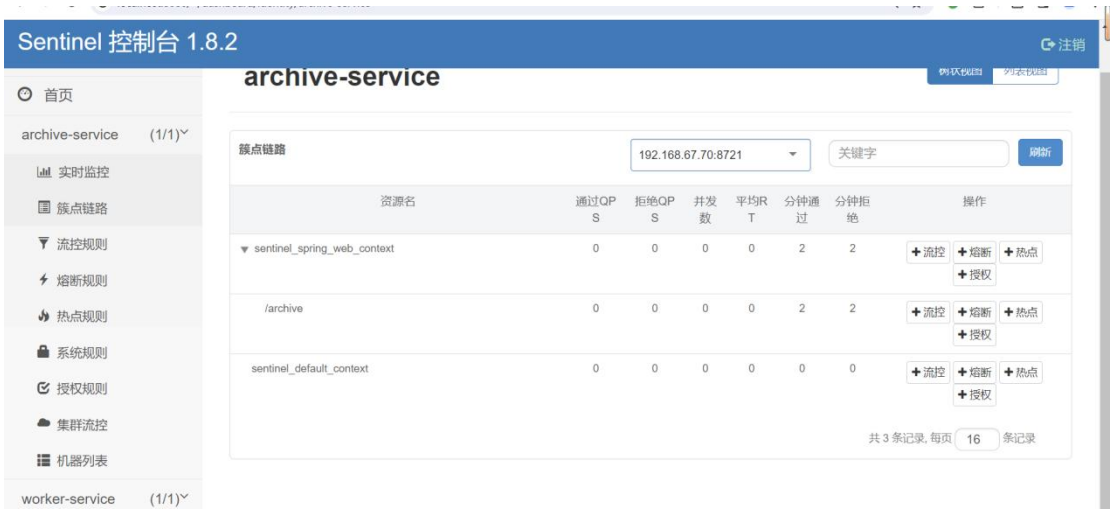
```
server:
  port: 8012

spring:
  application:
    name: sentinel-service

  cloud:
    nacos:
      discovery:
        server-addr: localhost:8848

    sentinel:
      transport:
        dashboard: localhost:8080
```

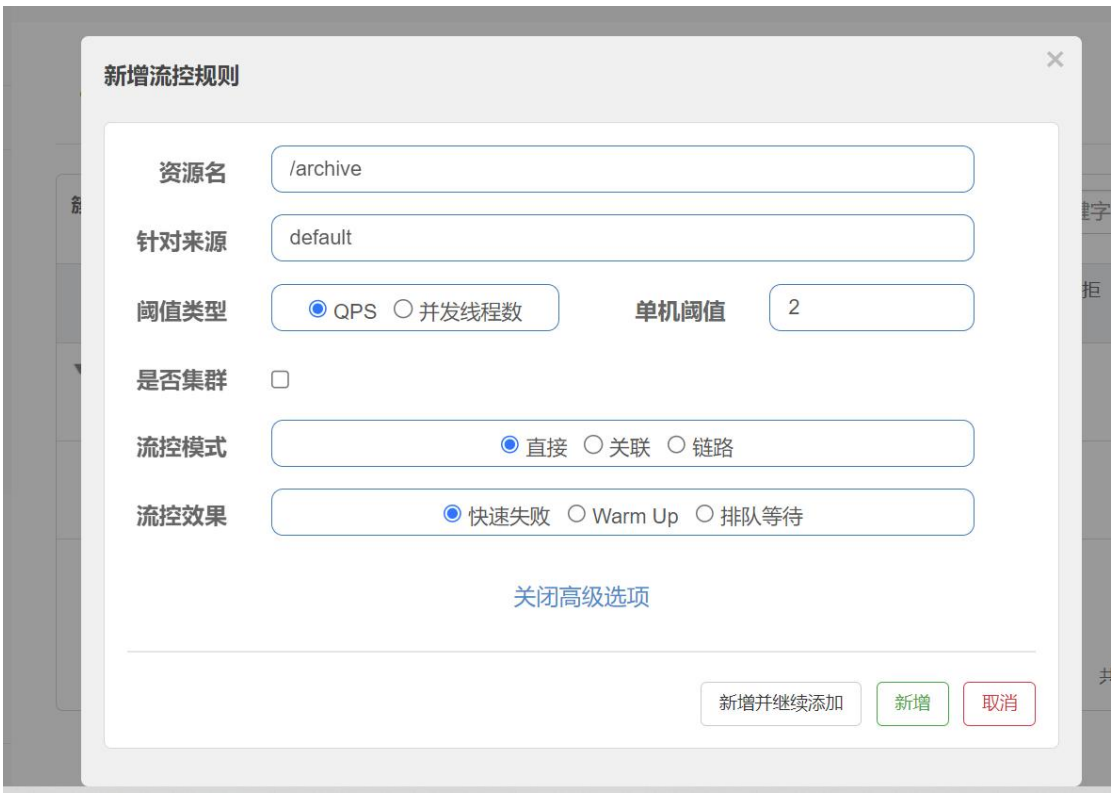
（6）访问类中的接口 http://localhost:8088/archive-serv/archive，触发 Sentinel 监控



3.5 体验基本的流控规则

（1）打开簇点链路

找到 /archive，点击“+流控”，配置流控规则



阈值类型 QPS　单机阈值 2

*：QPS 指的是每秒查询率

资源名：默认为操作节点的名称或 url。

针对来源：可以指定具体某个微服务掉用该节点时进行限流，默认为 default，表示对所有来源进行限流。

阈值类型：QPS 为每秒的请求次数，并发线程为该节点所使用的线程数。

流控模式：有三个，分别为直连、关联和链路。

直连：当接口达到限流条件时，进行限流。

关联：指定一个关联的接口，当关联的接口达到限流条件时进行限流。

链路：当从某个接口过来的资源达到限流条件时，开启限流。

流控效果：定义发生流控时的行为。

快速失败：当发生限流时，再接受到的请求直接返回失败信息。

Warm Up：冷启动，当微服务在同一时间收到大量请求时，不会直接让微服务达到最大阈值，而是有一个初始最大阈值，然后在指定时间内将初始最大阈值提升至最大阈值。初始最大阈值计算方法：总阈值 / coldFactor ，coldFactoy 的模式值为 3，可以在配置文件中更改

排队模式：超过阈值后，之后在每隔一段时间后（可以指定）才允许通过一个请求。场景：在某一秒有大量的请求到来，而接下来的几秒则处于空闲状态，我们希望系统能够在接下来的空闲期间逐渐处理这些请求，而不是在第一秒直接拒绝多余的请求。

（2）添加完流控规则后，测试。

快速刷新浏览器请求：http://localhost:8088/archive-serv/archive，观察实时流控。

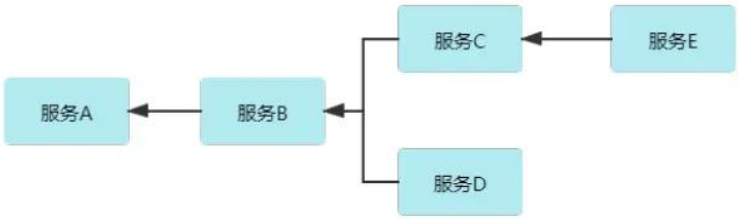
3.6 体验基本的熔断规则

什么是断路器和服务的熔断？

Sentinel 是微服务中的断路器，其作用为当某个服务因为压力过大或者宕机而变慢或者失效时，该服务的调用者会切断与该服务的链接，从而保护系统的可用性，这种策略叫做服务的熔断。

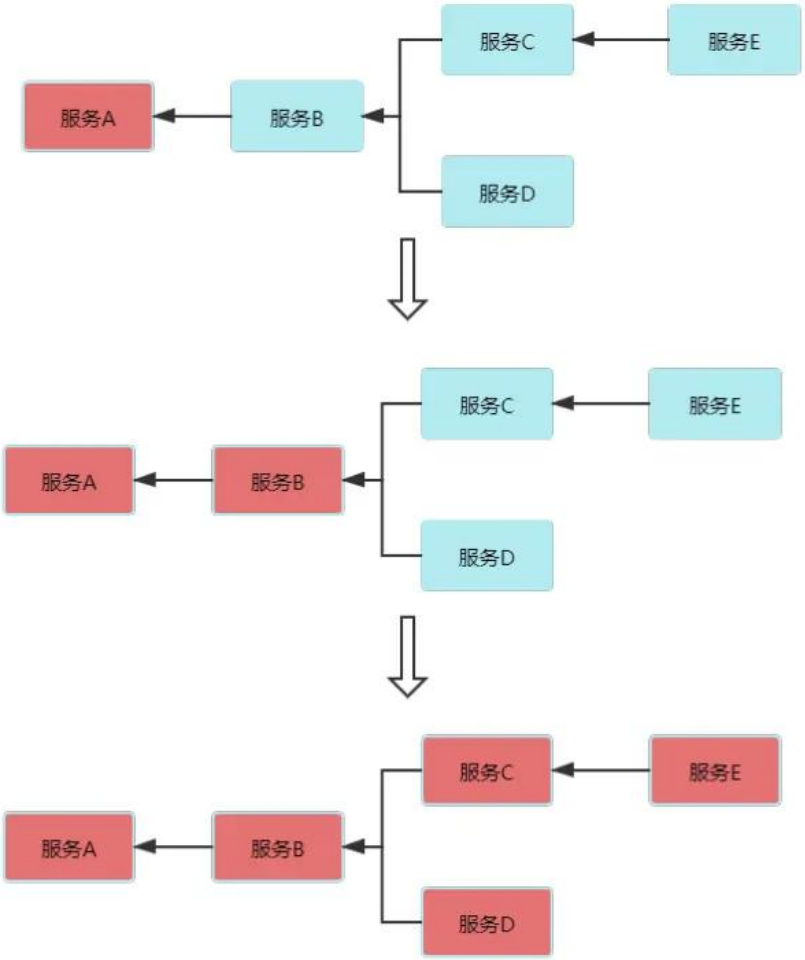
如果没有熔断会发生什么？

假如我们的系统中有 A、B、C、D、E 五个服务，其调用关系如图所示，B 调 A，D、C 调 B，E 调 C。



服务调用关系

如果没有熔断机制，当 A 服务因为某种因素发生了故障而导致该服务不可用，这时 B 服务调用 A 服务时，因为 A 服务的故障而导致访问 B 服务时，请求大量积压，从而拖慢 B 的速度，直至 B 的线程资源被耗尽，这时 B 服务也变的不可用，这种情况会在微服务系统中蔓延，拖垮整个系统，从而导致微服务的雪崩。



Sentinel 有三种熔断策略：慢调用比例、异常比例、异常数。

慢调用比例：选择以慢调用比例作为阈值，需要设置允许的慢调用 RT（即最大的响应时间），请求的响应时间大于该值则统计为慢调用。当单位统计时长（statIntervalMs）内请求数目大于设置的最小请求数目，并且慢调用的比例大于阈值，则接下来的熔断时长内请求会自动被熔断。经过熔断时长后熔断器会进入探测恢复状态（HALF-OPEN 状态），若接下来的一个请求响应时间小于设置的慢调用 RT 则结束熔断，若大于设置的慢调用 RT 则会再次被熔断。

（1）打开簇点链路

添加熔断策略

（2）给 archive 接口加个 sleep，模拟慢调用

```
Thread.sleep(2000);
```

（3）测试熔断

多次请求 http://localhost:8088/archive-serv/archive，会出现如下情况

Blocked by Sentinel (flow limiting)

等待超过熔断市场后，会恢复接口